

Name -> Manjot Singh
UID -> 23BAI70323
Section -> 23AML-3(B)
Subject -> Full stack Development

Assignment

Q.1 -> Summarize the benefits of using design patterns in frontend development.

Ans. -> Design patterns help developers build applications in a better and more organized way.

- They make code easy to reuse in different parts of the app
- Code becomes clean and easy to understand
- Applications can grow easily without breaking things
- Team members can work together smoothly
- Debugging and testing become simpler
- Development becomes faster because common problems already have solutions

Q.2 -> Classify the difference between global state and local state in React.

Ans. ->

Feature	Local State	Global State
Used in	Single component	Multiple components
Managed by	useState, useReducer	Redux, Context API
Lifetime	Only while component exists	Throughout the app
Best for	Forms, buttons, modals	Login, theme, user data
Complexity	Simple	More complex

Q.3 -> Compare different routing strategies in Single Page Applications (client-side, server-side, and hybrid) and analyse the trade-offs and suitable use cases for each.

Ans. -> **Client-side routing**

- Navigation handled in browser using JS
- Example: React Router

Pros

- Fast navigation
- Better UX
- Reduced server load

Cons

- SEO challenges
- Initial JS bundle size

Use cases

- Dashboards
- Authenticated apps

Server-side routing

- Each route served by server

Pros

- SEO-friendly
- Simple logic

Cons

- Slower navigation
- Higher server load

Use cases

- Content-heavy websites
- Blogs, CMS

Hybrid routing (SSR + CSR)

- Initial render on server, navigation on client

Pros

- Best SEO + performance
- Faster first paint

Cons

- Higher complexity
- Infrastructure cost

Use cases

- E-commerce
- Large-scale platforms

Q.4 -> Examine common component design patterns such as Container–Presentational, Higher-Order Components, and Render Props, and identify appropriate use cases for each pattern.

Ans. -> **4) Component design patterns**

Container–Presentational

- Container: logic, data
- Presentational: UI only

Use cases

- Clean separation of UI and business logic
- Large reusable UI components

Higher-Order Components (HOC)

- Function that wraps a component

Use cases

- Authentication
- Logging
- Access control

Render Props

- Component receives a function as child

Use cases

- Shared logic with dynamic UI
- Advanced customization

Q.5 -> Demonstrate and develop a responsive navigation bar using Material UI components while applying appropriate styling and breakpoint configurations.

Ans. ->

```
import React from "react";
import { AppBar, Toolbar, Typography, Button, IconButton, Box } from "@mui/material";
import MenuIcon from "@mui/icons-material/Menu";
```

```
export default function Navbar() {
  return (
    <AppBar position="static">
      <Toolbar>
        <IconButton
          edge="start"
          color="inherit"
          sx={{ display: { xs: "block", md: "none" } }}
        >
          <MenuIcon />
        </IconButton>

        <Typography variant="h6" sx={{ flexGrow: 1 }}>
          ProjectTool
        </Typography>

        <Box sx={{ display: { xs: "none", md: "flex" } }}>
          <Button color="inherit">Dashboard</Button>
          <Button color="inherit">Projects</Button>
          <Button color="inherit">Team</Button>
          <Button color="inherit">Profile</Button>
        </Box>
      </Toolbar>
    </AppBar>
  );
}
```

Q.6 -> Evaluate and design a complete frontend architecture for a collaborative project management tool with real-time updates. Include: a) SPA structure with nested routing and protected routes b) Global state management using Redux Toolkit with middleware c) Responsive UI design using Material UI with custom theming d) Performance optimization techniques for large datasets e) Analyse scalability and recommend improvements for multi-user concurrent access.

Ans. ->

a) SPA structure with routing

- /login
- /dashboard
 - /projects
 - /projects/:id
 - /settings

Protected routes using auth guards

b) Global state management

- Redux Toolkit slices:
 - authSlice

- projectSlice
 - userSlice
- Middleware:
 - Think for async actions
 - WebSocket middleware for real-time sync

c) Responsive UI with Material UI

- Custom theme:
 - Brand colors
 - Typography scale
- Layout:
 - Grid + Box
 - Breakpoints for mobile/tablet/desktop

d) Performance optimization for large datasets

- Virtualized lists
- Pagination and lazy loading
- Memoization (useMemo, useCallback)
- Normalized Redux state
- Code splitting with dynamic imports

e) Scalability and multi-user concurrency

Challenges

- State conflicts
- Real-time synchronization
- Performance bottlenecks

Recommendations

- WebSockets for live updates
- Optimistic UI updates
- Role-based access control
- Fine-grained permissions
- Event-based state updates
- Caching with stale-while-revalidate strategy